

Analysis of Algorithms

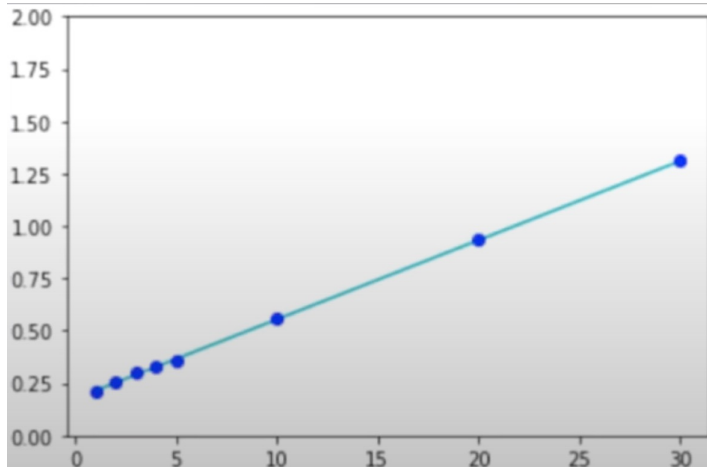
Discipline: Computer Science III

Overview

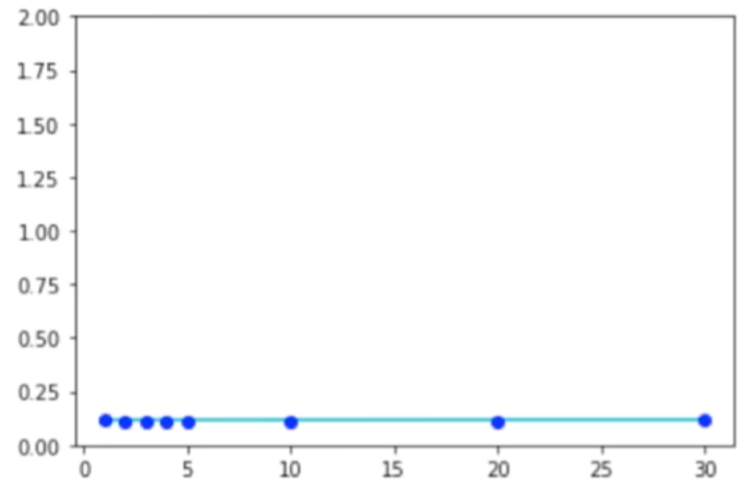
2

- Calculate the sum of all integer numbers from 1 up to (and including) some number n

```
public int addUpTo(int n)
{
    int total = 0;
    for (int i=1; i<=n; i++)
    {
        total += i;
    }
    return total;
}
```



```
public int addUpTo(int n)
{
    return n*(n+1)/2;
}
```



Scope

3

Analysis of Algorithms:

- Efficiency goals
- The concept of algorithm analysis
- Big-Oh notation
- The concept of asymptotic complexity
- Comparing various growth functions

Algorithm Efficiency

4

The efficiency of an algorithm is usually expressed in terms of its use of CPU time

The analysis of algorithms involves categorizing an algorithm in terms of efficiency

An everyday example: washing dishes

- Suppose washing a dish takes 30 seconds and drying a dish takes an additional 30 seconds
- Therefore, n dishes require n minutes ($60n$ seconds) to wash and dry

Algorithm Efficiency

5

Now consider a less efficient approach that requires us to dry all previously washed dishes each time we wash another one

Each dish takes 30 seconds to wash

But because we get the dishes wet while washing, we must dry the last dish once, the second last twice

- Dry time = $30 + 2*30 + 3*30 + \dots + (n-1)*30 + n*30$
 $= 30 * (1 + 2 + 3 + \dots + (n-1) + n)$

$$\text{time}(n \text{ dishes}) = n * (30 \text{ seconds wash time}) + \sum(i * 30)$$

$$\begin{aligned}\text{time}(n \text{ dishes}) &= 30n + \frac{30n(n+1)}{2} \\ &= 15n^2 + 45n \text{ seconds}\end{aligned}$$

Problem Size

6

For every algorithm we want to analyze, we need to define the size of the problem

The dishwashing problem has a size n

n = number of dishes to be washed/dried

For a search algorithm, the size of the problem is the size of the search pool

For a sorting algorithm, the size of the problem is the number of elements to be sorted

Problem Size

7

We must also decide what we are trying to efficiently optimize

- *time complexity* – CPU time
- *space complexity* – memory space

CPU time is generally the focus

A growth function shows the relationship between the size of the problem (n) and the value optimized (time)

Asymptotic Complexity

8

The growth function of the second dishwashing algorithm is

$$t(n) = 15n^2 + 45n$$

It is not typically necessary to know the exact growth function for an algorithm

We are mainly interested in the ***asymptotic complexity*** of an algorithm – the general nature of the algorithm as n increases

Asymptotic Complexity

9

Asymptotic complexity is based on the ***dominant term*** of the growth function – the term that increases most quickly as n increases

The dominant term for the second dishwashing algorithm is n^2 :

Number of dishes (n)	$15n^2$	$45n$	$15n^2 + 45n$
1	15	45	60
2	60	90	150
5	375	225	600
10	1,500	450	1,950
100	150,000	4,500	154,500
1,000	15,000,000	45,000	15,045,000
10,000	1,500,000,000	450,000	1,500,450,000
100,000	150,000,000,000	4,500,000	150,004,500,000
1,000,000	15,000,000,000,000	45,000,000	15,000,045,000,000
10,000,000	1,500,000,000,000,000	450,000,000	1,500,000,450,000,000

Big-Oh Notation

10

The coefficients and the lower order terms become increasingly less relevant as n increases

So we say that the algorithm is *order* n^2 , which is written $O(n^2)$

This is called *Big-Oh notation*

There are various Big-Oh categories

Two algorithms in the same category are generally considered to have the same efficiency, but that doesn't mean they have equal growth functions or behave exactly the same for all values of n

Big-Oh Categories

11

Some sample growth functions and their Big-Oh categories:

Growth Function	Order	Label
$t(n) = 17$	$O(1)$	constant
$t(n) = 3 \log n$	$O(\log n)$	logarithmic
$t(n) = 20n - 4$	$O(n)$	linear
$t(n) = 12n \log n + 100n$	$O(n \log n)$	$n \log n$
$t(n) = 3n^2 + 5n - 2$	$O(n^2)$	quadratic
$t(n) = 8n^3 + 3n^2$	$O(n^3)$	cubic
$t(n) = 2^n + 18n^2 + 3n$	$O(2^n)$	exponential

The Affection of CPU

12

You might think that faster processors would make efficient algorithms less important

A faster CPU helps, but not relative to the dominant term.

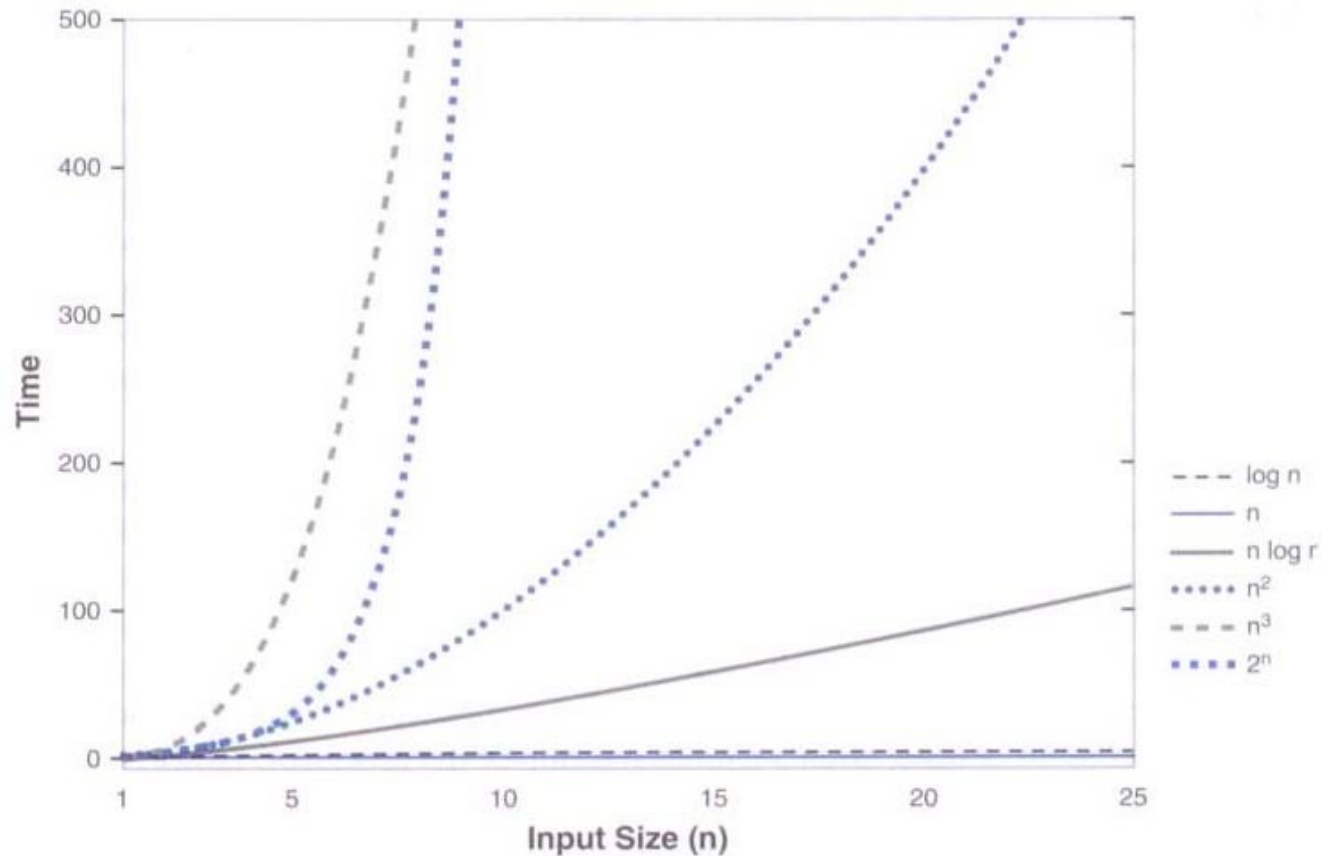
What happens if we increase our CPU speed by 10 times?

Algorithm	Time Complexity	Max Problem Size Before Speedup	Max Problem Size After Speedup
A	n	s_1	$10s_1$
B	n^2	s_2	$3.16s_2$
C	n^3	s_3	$2.15s_3$
D	2^n	s_4	$s_4 + 3.3$

Big-Oh Notation

13

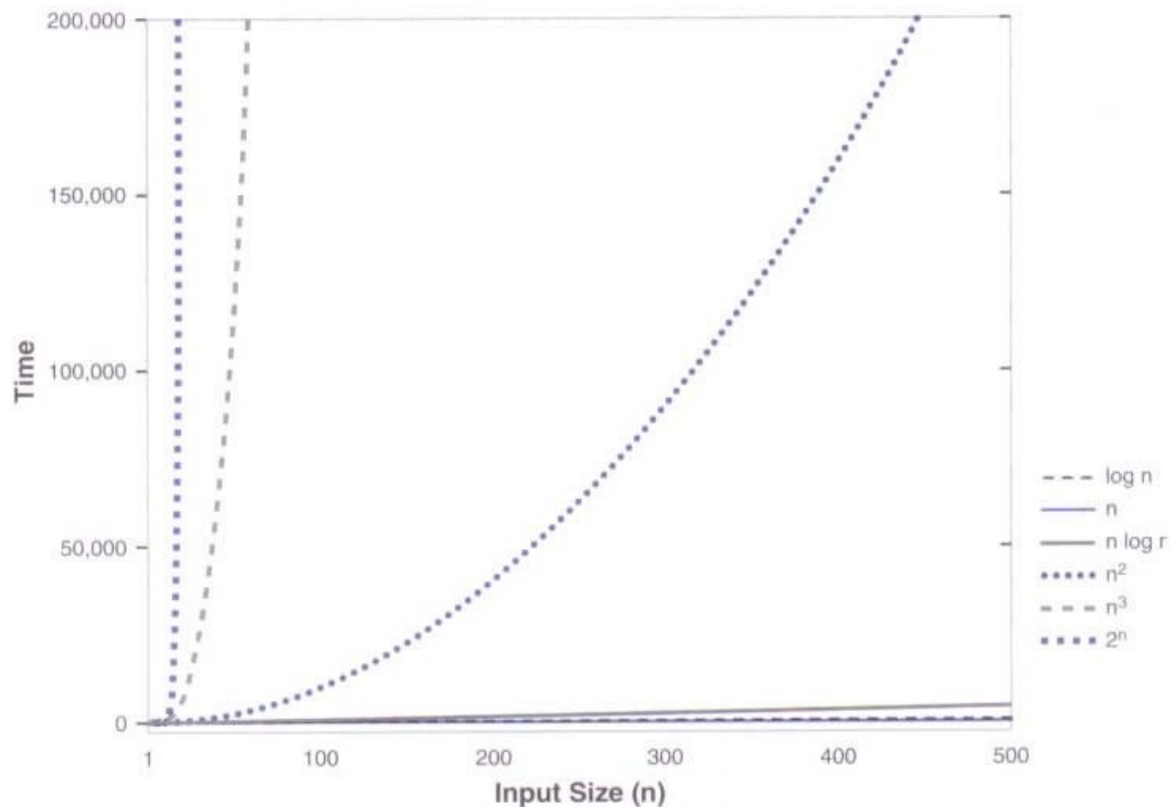
As n increases, the various growth functions diverge dramatically:



Big-Oh Notation

14

For large values of n , the difference is even more pronounced:



Analyzing Loop Execution

15

First determine the order of the body of the loop, then multiply that by the number of times the loop will execute

```
for (int count = 0; count < n; count++)  
    // some sequence of O(1) steps
```

N loop executions times O(1) operations results in a O(n) efficiency

Can write:

- CPU-time Complexity = $n * O(1)$
= $O(n*1)$
= $O(n)$

Analyzing Loop Execution

16

Consider the following loop:

```
count = 1;
while (count < n)
{
    count *= 2;
    // some sequence of O(1) steps
}
```

How often is the loop executed given the value of n ? The loop is executed $\log_2 n$ times, so the loop is $O(\log n)$

CPU-Time Efficiency = $\log n * O(1) = O(\log n)$

Analyzing Nested Loops

17

When loops are nested, we multiply the complexity of the outer loop by the complexity of the inner loop

```
for (int count = 0; count < n; count++)  
for (int count2 = 0; count2 < n; count2++)  
{  
    // some sequence of O(1) steps  
}
```

Both the inner and outer loops have complexity of $O(n)$ For Body has complexity of $O(1)$

$$\begin{aligned}\text{CPU-Time Complexity} &= O(n) * (O(n) * O(1)) \\ &= O(n) * (O(n * 1)) \\ &= O(n) * O(n) \\ &= O(n * n) = O(n^2)\end{aligned}$$

The overall efficiency is $O(n^2)$

Analyzing Method Calls

18

The body of a loop may contain a call to a method

To determine the order of the loop body, the order of the method must be taken into account

The overhead of the method call itself is generally ignored

Examples

19

```
public int addUpTo(int n)
{
    int total = 0;
    for (int i=1; i<=n; i++)
    {
        total += i;
    }
    return total;
}
```

```
public int addUpTo(int n)
{
    return n*(n+1)/2;
}
```

$O(1)$

$$O(1) + O(n) * O(1) + O(1) = O(n)$$

Examples

20

```
int x= 5+(15*20);  
for(int i=0;i<n;i++)  
{  
    System.out.println(i);  
}  
for(int i=0;i<n;i++)  
    for(int j=0;j<n;j++)  
    {  
        System.out.println(i*j);  
    }
```

Examples

21

```
for(int i=1;i <= Math.max(5,n);i++)  
{  
    System.out.println(i);  
}
```

```
for(int i=1;i <= Math.min(5,n);i++)  
{  
    System.out.println(i);  
}
```

Key Things to take away:

22

Algorithm Analysis:

- Software must make efficient use of resources such as CPU and memory
- Algorithm Analysis is an important fundamental computer science topic
- The order of an algorithm is found by eliminating constants and all but the dominant term in the algorithm's growth function
- When an algorithm is inefficient, a faster processor will not help
- Analyzing algorithms often focuses on analyzing loops
- Time complexity of a loop is found by multiplying the complexity of the loop body times the number of times the loop is executed.
- Time complexity for nested loops must multiply the inner loop complexity with the number of times through the outer loop